CYBEXER TECHNOLOGIES

Linux hardening

Linux hardening - lynis

Lynis is an auditing tool for hardening GNU/Linux and Unix based systems.

It scans the system configuration and creates an overview of system information and security issues.

Linux hardening - lynis

Lynis is available for most Linux operating systems from corresponding repositories, but very often it's not the latest version.

Let's install latest version of Lynis from Github

- **cd /opt**
- **git clone https://github.com/CISOfy/lynis**

```
root@ubuntu-2204:/etc/pam.d# cd /opt/
root@ubuntu-2204:/opt# git clone https://github.com/CISOfy/lynis
Cloning into 'lynis'...
remote: Enumerating objects: 15107, done.
remote: Counting objects: 100% (513/513), done.
remote: Compressing objects: 100% (205/205), done.
remote: Total 15107 (delta 345), reused 430 (delta 306), pack-reused 14594
Receiving objects: 100% (15107/15107), 8.04 MiB | 3.78 MiB/s, done.
Resolving deltas: 100% (11111/11111), done.
```

Linux hardening - lynis

After download, Lynis is ready to run. Firstly, let's verify you can execute it. Type following command in your terminal window:

- **cd lynis/**
- **./lynis**

```
[ Lynis 3.1.2 ]

################################################################################
  Lynis comes with ABSOLUTELY NO WARRANTY. This is free software, and you are
  welcome to redistribute it under the terms of the GNU General Public License.
  See the LICENSE file for details about using this software.

  2007-2021, CISOfy - https://cisofy.com/lynis/
  Enterprise support available (compliance, plugins, interface and tools)
################################################################################


[+] Initializing program
------------------------------------
```

Linux hardening - lynis

Now it's time to run system test

- **./lynis audit system**

```
[+] Initializing program
------------------------------------
  - Detecting OS...                                           [ DONE ]
  - Checking profiles...                                      [ DONE ]

  ---------------------------------------------------
  Program version:           3.1.2
  Operating system:          Linux
  Operating system name:     Ubuntu
  Operating system version:  22.04
  Kernel version:            5.15.0
  Hardware platform:         x86_64
  Hostname:                  ubuntu-2204
```

Linux hardening - lynis

After scan is finished - review the results.

```
[+] SSH Support
------------------------------------
  - Checking running SSH daemon                               [ FOUND ]
    - Searching SSH configuration                             [ FOUND ]
    - OpenSSH option: AllowTcpForwarding                      [ SUGGESTION ]
    - OpenSSH option: ClientAliveCountMax                     [ SUGGESTION ]
    - OpenSSH option: ClientAliveInterval                     [ OK ]
```

```
Lynis security scan details:

Hardening index : 64 [############        ]
Tests performed : 276
Plugins enabled : 2
```

```
* Consider hardening SSH configuration [SSH-7408]
  - Details  : MaxAuthTries (set 6 to 3)
    https://cisofy.com/lynis/controls/SSH-7408/

* Consider hardening SSH configuration [SSH-7408]
  - Details  : MaxSessions (set 10 to 2)
    https://cisofy.com/lynis/controls/SSH-7408/
```

```
Files:
- Test and debug information      : /var/log/lynis.log
- Report data                     : /var/log/lynis-report.dat
```

Linux hardening - lynis

Lynis allows to scan remote targets

- **./lynis audit system remote 192.168.1.100**

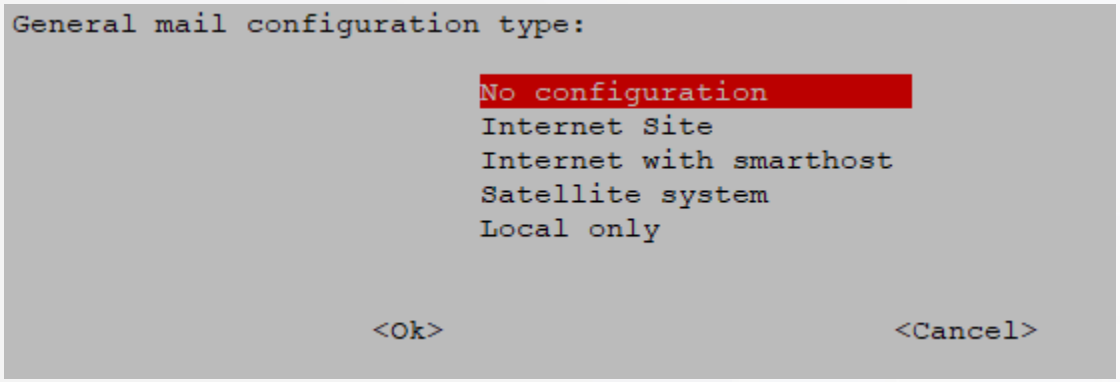
```
  How to perform a remote scan:
  ============================
  Target   : 192.168.1.100
  Command : ./lynis audit system

  * Step 1: Create tarball
    mkdir -p ./files && cd .. && tar czf ./lynis/files/lynis-remote.tar.gz --exclude=

  * Step 2: Copy tarball to target 192.168.1.100
    scp -q ./files/lynis-remote.tar.gz 192.168.1.100:~/tmp-lynis-remote.tgz

  * Step 3: Execute audit command
    ssh 192.168.1.100 "mkdir -p ~/tmp-lynis && cd ~/tmp-lynis && tar xzf ../tmp-lynis
audit system"

  * Step 4: Clean up directory
    ssh 192.168.1.100 "rm -rf ~/tmp-lynis"
```

Linux hardening - rkhunter

Rkhunter, aka Rootkit Hunter scans systems for known and unknown rootkits, backdoors, sniffers and exploits.

Let's install it on Ubuntu machine

- **apt-get install rkhunter**



Linux hardening - rkhunter

After **rkhunter** is installed on your system, let's run it

- **rkhunter --check**

```
  Performing 'shared libraries' checks
    Checking for preloading variables                    [ None found ]
    Checking for preloaded libraries                     [ None found ]
    Checking LD_LIBRARY_PATH variable                    [ Not found ]

  Performing file properties checks
    Checking for prerequisites                           [ OK ]
    /usr/sbin/adduser                                    [ OK ]
```

```
Checking for rootkits...

  Performing check of known rootkit files and directories
    55808 Trojan - Variant A                             [ Not found ]
    ADM Worm                                             [ Not found ]
    AjaKit Rootkit                                       [ Not found ]
    Adore Rootkit                                        [ Not found ]
```

Add '**--sk**' to skip keypress after each test

CYBEXER TECHNOLOGIES

SSH server hardening

Linux hardening - SSH server

SSH is a secure, encrypted replacement for common login services such as telnet, ftp, rlogin, rsh and rcp.

By default, on Linux systems, SSH configuration file is world-readable. Let's fix that mistake:

- **ls -la /etc/ssh/sshd_config**
- **chmod 600 /etc/ssh/sshd_config**
- **ls -la /etc/ssh/sshd_config**

```
root@ubuntu-2204:/opt/lynis# ls -al /etc/ssh/sshd_config
-rw-r--r-- 1 root root 3266 Dec 15  2022 /etc/ssh/sshd_config
root@ubuntu-2204:/opt/lynis# chmod 600 /etc/ssh/sshd_config
root@ubuntu-2204:/opt/lynis# ls -al /etc/ssh/sshd_config
-rw------- 1 root root 3266 Dec 15  2022 /etc/ssh/sshd_config
```

Linux hardening - SSH server

As with any other service, SSH service is not exception - before making any changes to the configuration, it's advised to created backup of existing configuration file

- **cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak**
- **ls -la /etc/ssh**

```
root@ubuntu-2204:~# cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
root@ubuntu-2204:~# ls -la /etc/ssh
total 564
drwxr-xr-x   4 root root   4096 Apr 11 11:37 .
drwxr-xr-x 104 root root   4096 Apr 11 10:34 ..
-rw-r--r--   1 root root 505426 Feb 26  2022 moduli
-rw-r--r--   1 root root   1650 Feb 26  2022 ssh_config
drwxr-xr-x   2 root root   4096 Feb 26  2022 ssh_config.d
-rw-------   1 root root   3266 Dec 15  2022 sshd_config
-rw-------   1 root root   3266 Apr 11 11:37 sshd_config.bak
```

Linux hardening - SSH server

Default SSH configuration usually is not the best one.

Let's see what options should/must be adjusted, to make SSH service more secure.

PermitRootLogin no - specifies whether root user can log in using ssh

MaxAuthTries 2 - specifies the maximum number of authentication attempts permitted per connection

Linux hardening - SSH server

Other SSH hardening settings:

LoginGraceTime 20 - the server disconnects after this time if the user has not successfully logged in

PasswordAuthentication no - specifies whether password authentication is allowed

PermitEmptyPasswords no - specifies whether the server allows login to accounts with empty password strings

Linux hardening - SSH server

Other SSH hardening settings:

MaxSessions 3 - desired maximum number of active sessions per user

Port 9022 - specifies the port number that sshd listens on

ListenAddress IPv4_address - specifies the local addresses sshd should listen on

Linux hardening - SSH server

Other SSH hardening settings:

X11Forwarding no - specifies whether X11 forwarding is permitted

AllowTcpForwarding no - specifies whether TCP forwarding is permitted

PermitTunnel no - specifies whether tun device forwarding is allowed

Linux hardening - SSH server

Before starting using updated SSH configuration, it is advised to verify SSH configuration file

- **sshd -t**

```
root@ubuntu-2204:~# sshd -t
/etc/ssh/sshd_config: line 33: Bad configuration option: PPermitRootLogin
/etc/ssh/sshd_config: terminating, 1 bad configuration options
root@ubuntu-2204:~#
```

Linux hardening - SSH server

If configuration check does not show any errors/warnings, then SSH service must be restarted to apply new settings

- **systemctl reload sshd.service**

Linux hardening - SSH server

For more tight permissions it is possible to set immutable bit to the configuration file

- **chattr +i /etc/ssh/sshd_config**

Verify immutable bit

- **lsattr /etc/ssh/sshd_config**

```
root@ubuntu-2204:/etc/ssh# lsattr /etc/ssh/sshd_config
----i---------e------- /etc/ssh/sshd_config
root@ubuntu-2204:/etc/ssh#
```

Remove immutable bit from file

- **chattr -i /etc/ssh/sshd_config**

CYBEXER TECHNOLOGIES

SSH key authentication

Linux hardening - ssh keys

SSH, or secure shell, is an encrypted protocol used to administer and communicate with servers.

An SSH server can authenticate clients using a variety of different methods. The most basic of these is password authentication, which is easy to use, but not the most secure.

Linux hardening - ssh keys

Although passwords are sent to the server in a secure manner, they are generally not complex or long enough to be resistant to repeated, persistent attackers. Modern processing power combined with automated scripts make brute-forcing a password-protected account very possible. Although there are other methods of adding additional security (fail2ban, etc.), SSH keys prove to be a reliable and secure alternative.

Linux hardening - ssh keys

SSH key pairs are two cryptographically secure keys that can be used to authenticate a client to an SSH server. Each key pair consists of a public key and a private key.

The private key is retained by the client and should be kept absolutely secret. Any compromise of the private key will allow the attacker to log into servers that are configured with the associated public key without additional authentication. As an additional precaution, the key can be encrypted on disk with a passphrase.

Linux hardening - ssh keys

Let's now create SSH key set.

Login to your Kali Linux server (**10.XX.3.5**) with SSH client and type following command:

- **ssh-keygen -a 100 -t ed25519 -f ~/.ssh/id_ed25519 -C "userXX"**

```
root@kali:~# ssh-keygen -a 100 -t ed25519 -f ~/.ssh/id_ed25519 -C "userXX"
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519
Your public key has been saved in /root/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:4pO8Fnhx9Jzlie9LbBLwRQE+Rsv0K9qmShhL9zNim2E userXX
The key's randomart image is:
+--[ED25519 256]--+
```

Linux hardening - ssh keys

Your SSH keys might use one of the following algorithms:

- 🚨 **DSA***:* It's unsafe and even no longer supported since OpenSSH version 7, you need to upgrade it!
- ⚠️ **RSA***:* It depends on key size. If it has 3072 or 4096-bit length, then you're good. Less than that, you probably want to upgrade it. The 1024-bit length is even considered unsafe.
- 👀 **ECDSA***:* It depends on how well your machine can generate a random number that will be used to create a signature. There's also a trustworthiness concern on the NIST curves that being used by ECDSA.
- ✅ **Ed25519***:* It's the most recommended public-key algorithm available today!

Linux hardening - ssh keys

SSH key generation takes care about proper file permissions.

Private key must be readable only to the owner of the key, while public key can have world-readable permissions.

Verify generated keys

- **ls -la ~/.ssh/id***

```
root@kali:~# ls -la ~/.ssh/id*
-rw------- 1 root root 399 Jun  7 21:08 /root/.ssh/id_ed25519
-rw-r--r-- 1 root root  88 Jun  7 21:08 /root/.ssh/id_ed25519.pub
root@kali:~#
```

Linux hardening - ssh keys

Let's print public key from our private and compare it to existing public key.

Verify generated keys

- **diff <(ssh-keygen -y -e -f /root/.ssh/id_ed25519) <(ssh-keygen -y -e -f /root/.ssh/id_ed25519.pub)**

If no output is shown, then keys belong together.

Linux hardening - ssh keys

Now, let's transfer key to another SSH server, where we want to enable passwordless authentication. Type following commands in Kali Linux terminal window:

- **ssh-copy-id -i ~/.ssh/id_ed25519.pub root@10.XX.1.10**

```
root@kali:~# ssh-copy-id -i ~/.ssh/id_ed25519.pub root@10.11.32.4
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_ed25519.pub"
The authenticity of host '10.11.32.4 (10.11.32.4)' can't be established.
ED25519 key fingerprint is SHA256:KRXQ6xpSIQQbOQZyyF4Xtce43njyJEWt0tAvqfvN+2w.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
root@10.11.32.4's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'root@10.11.32.4'"
and check to make sure that only the key(s) you wanted were added.
```

Linux hardening - ssh keys

After public key was transferred to remote machine, we can check ssh-key login. Type following command:

- **ssh -i ~/.ssh/id_ed25519 root@10.XX.1.10**

```
root@kali:~# ssh -i ~/.ssh/id_ed25519 root@10.11.32.4
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-73-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Wed Jun  7 09:22:56 PM EEST 2023

  System load:  0.078125          Processes:               214
  Usage of /:   61.4% of 19.51GB  Users logged in:         0
  Memory usage: 33%               IPv4 address for ens160: 10.11.32.4
  Swap usage:   0%                IPv6 address for ens160: fd18:c01:11:32::4

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.
```

Linux hardening - ssh keys

To make private key more secure, we can set passphrase to it. Run following command in you Kali Linux terminal window:

- **ssh-keygen -p -a 100 -f ~/.ssh/id_ed25519**

```
root@kali:~# ssh-keygen -p -a 100 -f ~/.ssh/id_ed25519
Key has comment 'userXX'
Enter new passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved with the new passphrase.
root@kali:~#
```

Linux hardening - ssh keys

Now, let's connect to remote server with passphrase protected private key:

- **ssh -i ~/.ssh/id_ed25519 root@10.XX.1.10**

```
root@kali:~# ssh -i ~/.ssh/id_ed25519 root@10.11.32.4
Enter passphrase for key '/root/.ssh/id_ed25519':
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-73-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Wed Jun  7 09:28:49 PM EEST 2023

  System load:  0.134765625       Processes:               216
  Usage of /:   61.4% of 19.51GB  Users logged in:         0
  Memory usage: 33%               IPv4 address for ens160: 10.11.32.4
  Swap usage:   0%                IPv6 address for ens160: fd18:c01:11:32::4
```

Linux hardening - ssh keys

To simplify login to remote system, without specifying every time a key file, there's an option to create a special 'config' file - **/root/.ssh/config**

Host web

HostName 10.XX.1.10

User root

Port 22

IdentityFile ~/.ssh/id_ed25519

Linux hardening - ssh keys

After all required information about remote machine is saved, you can login with following command:

- **ssh web**

```
root@kali:~# ssh graylog
Enter passphrase for key '/root/.ssh/id_ed25519':
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-73-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Wed Jun  7 09:35:47 PM EEST 2023

  System load:  0.005859375       Processes:               213
  Usage of /:   61.4% of 19.51GB  Users logged in:         0
  Memory usage: 34%               IPv4 address for ens160: 10.11.32.4
  Swap usage:   0%                IPv6 address for ens160: fd18:c01:11:32::4

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.
```

Linux hardening - ssh keys

Once ssh key login to the remote system is verified, it's time to disable password authentication in SSH server.

Login to Linux server (**10.XX.1.10**) and run following command:

- **sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config**
- **sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config**

Linux hardening - ssh keys

After SSH server configuration modification, restart SSH server

- **systemctl restart sshd**

Now, from Kali Linux terminal try to login to web server with 'ubuntu' user

- **ssh ubuntu@10.XX.1.10**

```
root@kali:~# ssh ubuntu@10.11.32.4
ubuntu@10.11.32.4: Permission denied (publickey).
root@kali:~#
```

Linux hardening - ssh keys

And next step is to login from Kali Linux terminal to login to web server with 'root' user

- **ssh -i ~/.ssh/id_ed25519 root@10.XX.1.10**

Linux hardening - SSH server

SSH default configuration uses weak/obsolete ciphers and key exchange algorithms. To see what can be hardened and fixed, let's use ssh-audit tool. Install the tool on your Kali machine (**10.XX.1.10**)

- **apt-get install ssh-audit**

```
# apt-get install ssh-audit
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  ssh-audit
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 66.3 kB of archives.
After this operation, 397 kB of additional disk space will be used.
```

Linux hardening - SSH server

Let's now run ssh-audit from Kali machine (**10.XX.3.5**) against Ubuntu server (**10.XX.1.10**)

- **ssh-audit -p 22 10.XX.1.10**

```
# key exchange algorithms
(kex)  curve25519-sha256                      -- [info] available since OpenSSH 7.4, Dropbear SSH 2018.76
                                              `- [info] default key exchange since OpenSSH 6.4
(kex)  curve25519-sha256@libssh.org           -- [info] available since OpenSSH 6.4, Dropbear SSH 2013.62
                                              `- [info] default key exchange since OpenSSH 6.4
(kex)  ecdh-sha2-nistp256                     -- [fail] using elliptic curves that are suspected as being bac
                                              `- [info] available since OpenSSH 5.7, Dropbear SSH 2013.62
(kex)  ecdh-sha2-nistp384                     -- [fail] using elliptic curves that are suspected as being bac
                                              `- [info] available since OpenSSH 5.7, Dropbear SSH 2013.62
(kex)  ecdh-sha2-nistp521                     -- [fail] using elliptic curves that are suspected as being bac
                                              `- [info] available since OpenSSH 5.7, Dropbear SSH 2013.62
(kex)  sntrup761x25519-sha512@openssh.com     -- [info] available since OpenSSH 8.5
(kex)  diffie-hellman-group-exchange-sha256 (3072-bit) -- [info] available since OpenSSH 4.4
```

```
# algorithm recommendations (for OpenSSH 8.9)
(rec)  -ecdh-sha2-nistp256                    -- kex algorithm to remove
(rec)  -ecdh-sha2-nistp384                    -- kex algorithm to remove
(rec)  -ecdh-sha2-nistp521                    -- kex algorithm to remove
(rec)  -ecdsa-sha2-nistp256                   -- key algorithm to remove
(rec)  -hmac-sha1                             -- mac algorithm to remove
(rec)  -hmac-sha1-etm@openssh.com             -- mac algorithm to remove
(rec)  -diffie-hellman-group14-sha256         -- kex algorithm to remove
(rec)  -hmac-sha2-256                         -- mac algorithm to remove
(rec)  -hmac-sha2-512                         -- mac algorithm to remove
```

Linux hardening - SSH server

Now, let's adjust some cryptographic options of our SSH server

- **echo 'HostKeyAlgorithms ssh-ed25519-cert-v01@openssh.com,ssh-ed25519' >> /etc/ssh/sshd_config**
- **echo 'KexAlgorithms curve25519-sha256' >> /etc/ssh/sshd_config**
- **echo 'Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com' >> /etc/ssh/sshd_config**
- **echo 'MACs hmac-sha2-512-etm@openssh.com' >> /etc/ssh/sshd_config**

Linux hardening - SSH server

After restarting SSH server, re-run ssh-audit and see results

- **systemctl restart sshd**
- **ssh-audit -p 22 10.XX.1.10**

```
# general
(gen) banner: SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.6
(gen) software: OpenSSH 8.9p1
(gen) compatibility: OpenSSH 7.4+, Dropbear SSH 2018.76+
(gen) compression: enabled (zlib@openssh.com)

# key exchange algorithms
(kex) curve25519-sha256                -- [info] available since OpenSSH 7.4, Dropbear SSH 20
                                       `- [info] default key exchange since OpenSSH 6.4
(kex) kex-strict-s-v00@openssh.com     -- [info] pseudo-algorithm that denotes the peer suppo
tack (CVE-2023-48795)

# host-key algorithms
(key) ssh-ed25519                      -- [info] available since OpenSSH 6.5

# encryption algorithms (ciphers)
(enc) chacha20-poly1305@openssh.com    -- [info] available since OpenSSH 6.5
                                       `- [info] default cipher since OpenSSH 6.9
(enc) aes256-gcm@openssh.com           -- [info] available since OpenSSH 6.2
```

CYBEXER TECHNOLOGIES

Mod Security (aka web firewall)

Linux hardening - mod_security

ModSecurity is a free and open-source firewall tool supported by various web servers, such as Apache, Nginx, and IIS.

It is a signature-based firewall that is capable to block several types of attacks including, cross-site scripting (XSS), brute force attacks, and known code injection attacks.

It provides different rule sets that allow you to customize and configure your server security.

Linux hardening - mod_security

Login to your Linux server (**10.XX.1.10**).

Be sure your system is up to date.

- **apt-get -y update && apt-get -y upgrade**

```
root@graylog-snort:~# apt-get -y update && apt-get -y upgrade
Hit:1 http://ee.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://ee.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://ee.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 http://ee.archive.ubuntu.com/ubuntu jammy-security InRelease
Ign:5 https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/6.0 InRelease
Hit:6 https://artifacts.opensearch.org/releases/bundle/opensearch/2.x/apt stable InRelease
Hit:7 https://artifacts.elastic.co/packages/7.x/apt stable InRelease
Hit:8 https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/6.0 Release
Hit:10 https://packages.graylog2.org/repo/debian stable InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Linux hardening - mod_security

ModSecurity can be installed 2 ways: from source code and from Linux's repository.

Simplest way is to install from repository, which will automatically install and configure all dependent software. Run following in your terminal:

- **apt-get install apache2 libapache2-mod-security2**

```
root@graylog-snort:/etc# apt-get install libapache2-mod-security2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  liblua5.1-0 libyajl2 modsecurity-crs
Suggested packages:
  lua geoip-database-contrib ruby python
The following NEW packages will be installed:
  libapache2-mod-security2 liblua5.1-0 libyajl2 modsecurity-crs
0 upgraded, 4 newly installed, 0 to remove and 4 not upgraded.
Need to get 525 kB of archives.
After this operation, 2,440 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

Linux hardening - mod_security

On some Linux versions you need to enable mod_security Apache module.

Let's do it:

- **a2enmod security2**

```
root@web:~# a2enmod security2
Considering dependency unique_id for security2:
Enabling module unique_id.
Enabling module security2.
To activate the new configuration, you need to run:
  systemctl restart apache2
```

Linux hardening - mod_security

Next step is to verify ModSecurity module is loaded by Apache web server. Run following command in your terminal:

- **apachectl -M | grep security**

```
root@graylog-snort:~# apachectl -M | grep security
 security2_module (shared)
root@graylog-snort:~#
```

Linux hardening - mod_security

ModSecurity engine needs rules to work. The rules decide how communication is handled on the web server. Depending on the configuration, ModSecurity can pass, drop, redirect or execute a script.

Linux hardening - mod_security

There is a default configuration file **/etc/modsecurity/modsecurity.conf-recommended** which you should copy to **/etc/modsecurity/modsecurity.conf** to enable and configure ModSecurity. To do this, run the command below:

- **cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf**

Linux hardening - mod_security

By default, ModSecurity engine runs in detection mode. To enable protection mode, run following command in Linux terminal

- **sed -i 's/SecRuleEngine DetectionOnly/SecRuleEngine On/' /etc/modsecurity/modsecurity.conf**

Temporarily disable ModSecurity rules:

- **mv /usr/share/modsecurity-crs/owasp-crs.load /usr/share/modsecurity-crs/owasp-crs.load.temp**

Linux hardening - mod_security

After enabling new configuration, Apache web server must be restarted. Run following command to do that:

- **systemctl restart apache2**

To be sure, that ModSecurity components are loaded, check '**/var/log/apache2/**' folder:

- **ls -al /var/log/apache2**

```
root@graylog-snort:/etc/modsecurity# ls -al /var/log/apache2
total 552
drwxr-x---  2 root adm      4096 Jun  8 18:39 .
drwxrwxr-x 14 root syslog   4096 Jun  8 00:00 ..
-rw-r-----  1 root adm         0 Jun  8 00:00 access.log
-rw-r-----  1 root adm      4331 Jun  7 11:55 access.log.1
-rw-r-----  1 root adm       224 Jun  6 16:11 access.log.2.gz
-rw-r-----  1 root adm     12948 Jun  8 18:39 error.log
-rw-r-----  1 root adm      2904 Jun  8 00:00 error.log.1
-rw-r-----  1 root adm       652 Jun  7 00:07 error.log.2.gz
-rw-r-----  1 root root        0 Jun  8 18:39 modsec_audit.log
-rw-r-----  1 root adm    374925 Jun  8 09:49 other_vhosts_access.log
```

Linux hardening - mod_security

Next step is to install PHP on the server. Run following commands in your terminal window:

- **apt-get -y install software-properties-common**
- **add-apt-repository ppa:ondrej/php -y**
- **apt-get -y install libapache2-mod-php7.1**

```
root@graylog-snort:~# apt-get -y install libapache2-mod-php7.1
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  php-common php7.1-cli php7.1-common php7.1-json php7.1-opcache php7.1-readline
Suggested packages:
  php-pear
The following NEW packages will be installed:
```

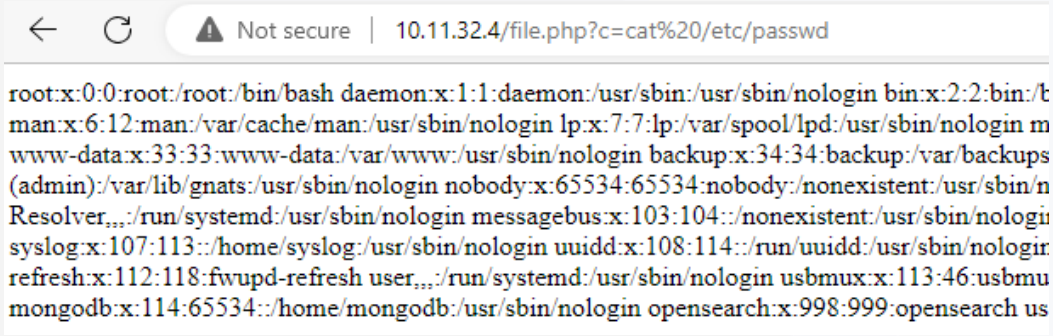
Linux hardening - mod_security

Let's create simple PHP backdoor

- **echo '<?php system($_GET[c]); ?>' > /var/www/html/file.php**

Verify, your backdoor is working. Open following link in your browser

- **http://10.XX.1.10/file.php?c=cat /etc/passwd**



Linux hardening - mod_security

Let's create simple ModSecurity rule

- **echo 'SecRule ARGS_GET "@contains passwd" "id:1,phase:1,t:lowercase,deny"' >> /etc/modsecurity/local_rules.conf**

After adding new rule, Apache web server must be restarted

- **service apache2 restart**

Linux hardening - mod_security

Refresh your browser window or open following link in your browser:

- **http://10.XX.1.10/file.php?c=cat /etc/passwd**

Linux hardening - mod_security

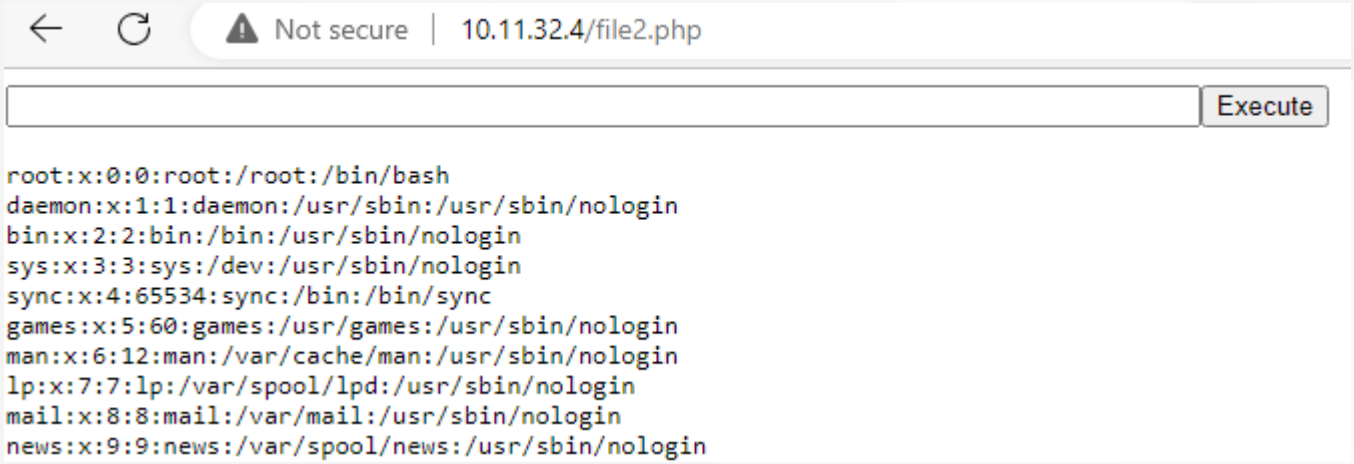
Now, let's create PHP shell, which executes commands in POST mode. Run following command in your terminal:

- **echo '<form method="POST" name="file2.php"><input type="TEXT" name="cmd" autofocus id="cmd" size="80"><input type="SUBMIT" value="Execute"></form><pre><?php if(isset($_POST[cmd])) { system($_POST[cmd]); } ?></pre>' > /var/www/html/file2.php**

Linux hardening - mod_security

Open following link in your browser and execute some Linux system command (e.g., **cat /etc/passwd**):

- **http://10.XX.1.10/file2.php**



Linux hardening - mod_security

Let's create ModSecurity rule, which intercepts POST requests:

- **echo 'SecRule ARGS_POST "@contains passwd" "id:2,phase:2,t:lowercase,deny,log,msg:PHP_passwd_attack"' >> /etc/modsecurity/local_rules.conf**

After adding new rule, Apache web server must be restarted

- **service apache2 restart**

Linux hardening - mod_security

Open following link in your browser and execute some Linux system command (e.g., **cat /etc/passwd**):

- **http://10.XX.1.10/file2.php**

Linux hardening - mod_security

Basic format of ModSecurity rule:

SecRule VARIABLES OPERATOR [ACTIONS]

Following variables may be used:

- **ARGS_GET** contains only query string parameters.
- **ARGS_POST** contains arguments from the POST body.
- **QUERY_STRING** Contains the query string part of a request URI. The value in QUERY_STRING is always provided raw, without URL decoding taking place.

Linux hardening - mod_security

Following variables may be used:

- **REQUEST_HEADERS** This variable can be used as either a collection of all of the request headers or can be used to inspect selected headers.
- **REQUEST_METHOD** This variable holds the request method used in the transaction.
- **REQUEST_URI** This variable holds the full request URL including the query string data (e.g., /index.php?p=X).

Linux hardening - mod_security

OPERATORS

This specifies a regular expression, pattern or keyword to be checked in the variable(s). Operators begin with the @ character. Example of operators:

- **@beginsWith**
- **@contains**
- **@endsWith**
- **@eq, @ge, @le**
- **@rx**

Linux hardening - mod_security

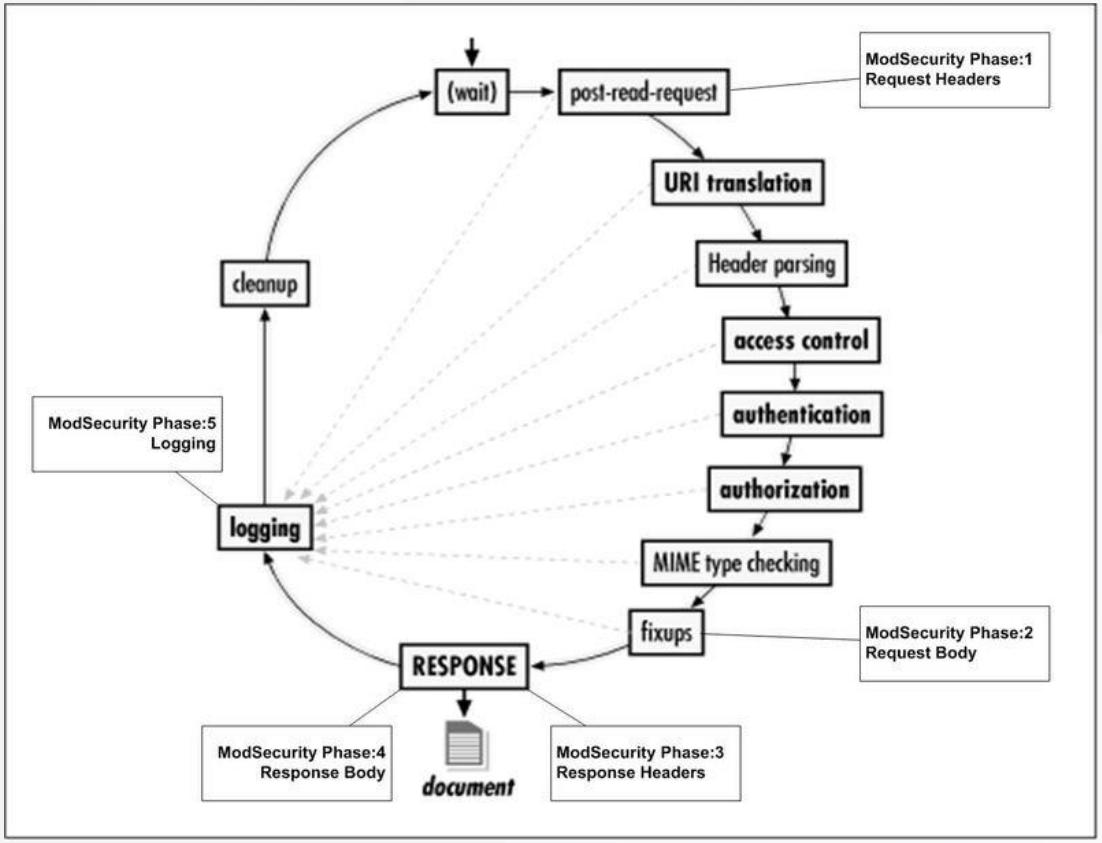
ACTIONS - specifies what to do if the rule matches.

Disruptive actions used to allow ModSecurity to take an action, for example allow or block.

Meta-data actions are used to provide more information about rules. Examples include id, rev, severity and msg.

Linux hardening - mod_security

Each rule must have a phase.



Linux hardening - mod_security

By default, all matched rules logs go to **/var/log/apache2/modsec_audit.log**

log - rule matches appear in both the error logs (**/var/log/apache2/error.log)** and audit logs (**/var/log/apache2/modsec_audit.log**).

msg:PHP_passwd_attack - custom message assigned to the rule.

CYBEXER TECHNOLOGIES

PAM and other

Linux hardening - PAM

PAM (Pluggable Authentication Modules) is a service that implements modular authentication modules on UNIX systems.

PAM is implemented as a set of shared objects that are loaded and executed when a program needs to authenticate a user.

Linux hardening - PAM

The **pam_pwquality.so** module checks the strength of passwords. It performs checks such as making sure a password is not a dictionary word, it is a certain length, contains a mix of characters (e.g. alphabet, numeric, other) and more. If module is missing, then install it with following command

- **apt-get install libpam-pwquality**

```
root@ubuntu-2204:~# apt-get install libpam-pwquality
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libbit-vector-perl libcarp-clan-perl libdate-calc-perl libdate-calc-xs-perl libiptables-c
  libnetaddr-ip-perl libsocket6-perl libunix-syslog-perl whois
Use 'apt autoremove' to remove them.
The following additional packages will be installed:
  cracklib-runtime libcrack2 libpwquality-common libpwquality1 wamerican
The following NEW packages will be installed:
  cracklib-runtime libcrack2 libpam-pwquality libpwquality-common libpwquality1 wamerican
0 upgraded, 6 newly installed, 0 to remove and 0 not upgraded.
Need to get 448 kB of archives.
```

Linux hardening - PAM

Main configuration file located at "**/etc/security/pwquality.conf**".

Some options to increase password requirements

minlen = 12 (password must be over 12 characters)

minclass = 4 (password will use 4 classes - digits, uppercase, lowercase, others)

dictcheck = 1 (check for the words from the cracklib dictionary)

enforcing = 1 (password is rejected if it fails the check)

Linux hardening - PAM

To ensure password reuse is limited, "**remember=5**" must be added to "**/etc/pam.d/common-password**" file:

- **common-password:password [success=1 default=ignore] pam_unix.so obscure use_authtok try_first_pass yescrypt remember=5**

Linux hardening - other

To ensure system accounts are secured, check for shell field in "/etc/passwd" file

- **awk -F: '{print $1,$7}' /etc/passwd**

```
root /bin/bash
daemon /usr/sbin/nologin
bin /usr/sbin/nologin
sys /usr/sbin/nologin
sync /bin/sync
games /usr/sbin/nologin
man /usr/sbin/nologin
lp /usr/sbin/nologin
mail /usr/sbin/nologin
news /usr/sbin/nologin
uucp /usr/sbin/nologin
```

To change login shell for user run following command

- **usermod -s $(which nologin) user_name**

Linux hardening - other

To ensure only "root" user has UID=0, run following command

- **awk -F: '$3 == "0" {print $1}' /etc/passwd**

If you see usernames except 'root', then you must take it very seriously and fix the issue.

Linux hardening - other

Data in world-writable files can be modified and compromised by any user on the system. World writable files may also indicate an incorrectly written script or program that could potentially be the cause of a larger compromise to the system's integrity.

Let's search for world-writable files

- **df --local -P | awk '{if (NR!=1) print $6}' | xargs -I '{}' find '{}' -xdev -type f -perm -0002**

Linux hardening - other

Sometimes when administrators delete users from the password file, they neglect to remove all files owned by those users from the system.

A new user who is assigned the deleted user's user ID or group ID may then end up "owning" these files, and thus have more access on the system than was intended.

Let's search for files without users

- **df --local -P | awk {'if (NR!=1) print $6'} | xargs -I '{}' find '{}' -xdev -nouser**

"-nogroup" - search for missing GIDs

Linux hardening - other

The "**noexec**" mount option specifies that the filesystem cannot contain executable binaries.

- **cat /etc/fstab**

```
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point>   <type>  <options>       <dump>  <pass>
# / was on /dev/sda2 during curtin installation
/dev/disk/by-uuid/3664b8a6-c951-4817-a515-c52f00042ee2 / ext4 defaults 0 1
/swap.img       none    swap    sw      0       0
```

Options to disable execution of binary files on "/tmp"

- **/tmp tmpfs tmpfs rw,noexec,relatime,seclabel**

Linux hardening - other

On production or critical Linux systems, it is advised not to have compilers.

If for some reasons compiler is need, then execution permissions must be tightened.

Let's see current compiler permissions:

- **which {gcc,cc,make} | xargs ls -la {}**

```
ls: cannot access '{}': No such file or directory
lrwxrwxrwx 1 root root     20 Mar 27 10:43 /usr/bin/cc -> /etc/alternatives/cc
lrwxrwxrwx 1 root root      6 Aug  5  2021 /usr/bin/gcc -> gcc-11
-rwxr-xr-x 1 root root 255696 Feb 15  2022 /usr/bin/make
```

Linux hardening - other

To monitor integrity of the file system, tool "**aide**" can be used. AIDE takes a snapshot of filesystem state including modification times, permissions, and file hashes which can then be used to compare against the current state of the filesystem to detect modifications to the system.

Let's install it on Ubuntu (**10.XX.1.10**)

- **apt-get install -y aide aide-common**

```
root@ubuntu-2204:/tmp# apt-get install -y aide aide-common
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libbit-vector-perl libcarp-clan-perl libdate-calc-perl libdate-calc-xs-perl l
  libnetaddr-ip-perl libsocket6-perl libunix-syslog-perl whois
Use 'apt autoremove' to remove them.
The following additional packages will be installed:
  libmhash2
Suggested packages:
  figlet
```

Linux hardening - other

Brief overview of available options

```
# These are the default rules.
#
#p:      permissions
#i:      inode:
#n:      number of links
#u:      user
#g:      group
#s:      size
#b:      block count
#m:      mtime
#a:      atime
#c:      ctime
#S:      check for growing size
#acl:            Access Control Lists
#selinux         SELinux security context
#xattrs:         Extended file attributes
#md5:    md5 checksum
#sha1:   sha1 checksum
#sha256:         sha256 checksum
#sha512:         sha512 checksum
#rmd160: rmd160 checksum
#tiger:  tiger checksum

#haval:  haval checksum (MHASH only)
#gost:   gost checksum (MHASH only)
#crc32:  crc32 checksum (MHASH only)
#whirlpool:      whirlpool checksum (MHASH only)

#R:              p+i+n+u+g+s+m+c+acl+selinux+xattrs+md5
#L:              p+i+n+u+g+acl+selinux+xattrs
#E:              Empty group
#>:              Growing logfile p+u+g+i+n+S+acl+selinux+xattrs
```

Linux hardening - other

Firstly, "aide" needs to be initialized. Let's run following command in Ubuntu terminal (it might take some time):

- **aideinit**
- **mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db**

```
AIDE initialized database at /var/lib/aide/aide.db.new
Ignored e2fs attributes: EIh

Number of entries:      0

---------------------------------------------------
The attributes of the (uncompressed) database(s):
---------------------------------------------------

/var/lib/aide/aide.db.new
 SHA256    : gEqM1rUER1NwTAV19VzBGY1Iv0F1IUJ2
             o11k4NNWhEQ=
 SHA512    : nZY0xRn6DzGm7z4EsZcUlqt5dWI1SxY8
             At/weoNpUJdlK6EpnOKpiiHOELhbiHTT
             zV+7mXT76OL4dKe7fiWHzw==
 RMD160    : bsXJz3nRPzhhutzPUFtipLorO8Q=
```

Linux hardening - other

Let's do now some modifications in "/root" folder

- **mkdir /root/folder_1**
- **echo "Hello" > /root/folder_1/file1.txt**
- **mkdir -p /root/bin**
- **cp /usr/bin/gcc-11 /root/bin/**

Linux hardening - other

Now let's run "aide" with check command

- **aide -c /etc/aide/aide.conf --check**

```
Summary:
  Total number of entries:      157194
  Added entries:                3
  Removed entries:              1
  Changed entries:              1

---------------------------------------------------
Added entries:
---------------------------------------------------

d++++++++++++++++++: /root/folder_1
f++++++++++++++++++: /root/folder_1/file1.txt
f++++++++++++++++++: /var/lib/aide/aide.db
```

To update DB run following command

- **aide -c /etc/aide/aide.conf --update**

CYBEXER TECHNOLOGIES

IPTABLES (aka Linux firewall)

Iptables - description

Iptables is a user-space application program that allows system administrator to configure tables provided by the Linux kernel firewall (implemented as different Netfilter modules) and the chains and rules it stores

Netfilter is a framework inside Linux kernel which offers flexibility for various networking-related operations to be implemented in form of customized handlers

offers various options for packet filtering, network address translation, and port translation

these functions provide the functionality required for directing packets through a network, as well as for providing ability to prohibit packets from reaching sensitive locations within a computer network

Iptables - actions

ACCEPT

- iptables stops further processing
- The packet is handed over to the end application or the operating system for processing

DROP

- iptables stops further processing
- The packet is blocked

LOG

- The packet information is sent to the syslog daemon for logging
- iptables continues processing with the next rule in the table
- You can't log and drop at the same time ->use two rules

REJECT

- Works like the DROP target, but will also return an error message to the host sending the packet that the packet was blocked

Iptables - basic usage

Check current firewall rules

- **iptables -L -nv**

```
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
```

Note. For IPv6 rules, you must use '**ip6tables**' command

Iptables - basic usage

Check current firewall rules and show their numbers

- **iptables -L -nv --line-numbers**

For IPv6 rules use following command

- **ip6tables -L -nv --line-numbers**

Iptables - basic usage

Let's create simple iptables rule, which blocks incoming 'ping' request

- **iptables -A INPUT -p icmp --icmp-type echo-request -j DROP**

Now try to 'ping' your machine from remote server

And try to 'ping' from your machine

Iptables - basic usage

Let's create simple iptables rule, which blocks outgoing 'ping' requests

- **iptables -A OUTPUT -p icmp --icmp-type echo-request -j DROP**

Try to 'ping' remote host from your machine

Iptables - basic usage

To delete firewall rule, use '**-D**' instead of '**-A**'

- **iptables -D INPUT -p icmp --icmp-type echo-request -j DROP**
- **iptables -D OUTPUT -p icmp --icmp-type echo-request -j DROP**

Iptables - basic usage

Very often, you need to specify exact source or destination IP address or network range. Use '-s' or '-d' option followed by desired IP address

- **iptables -A OUTPUT -p tcp -d 46.226.143.34 -j DROP**

- **iptables -A INPUT -p tcp -s 192.168.10.11 -j DROP**

You can use CIDR notations for networks

Iptables - basic usage

If server has several network interfaces, you may set specific interface name with '**-i**' option followed by interface name

- **iptables -A INPUT -i wlan0 -p icmp -j DROP**

Iptables - basic usage

For more precise filtering, '**iptables**' allows to set source and/or destination ports.

- **iptables -A INPUT -p tcp --dport 22 -j ACCEPT**

- **iptables -A OUTPUT -p tcp -d 192.162.1.0/24 --sport 21 -j ACCEPT**

For setting port ranges use '**1520:1540**' with '--sport' or '--dport' options. Or you can comma-separate ports: **22,80,443**

Iptables - basic usage

Since 'iptables' firewall processing rules in strict order (from top to bottom), you must be careful with IP address exceptions. E.g., you want to block all SSH traffic from whole network, except 1 IP address.

Correct way is to create one 'accept' rule from single IP address. And second rule will block whole network.

- **iptables -A INPUT -p tcp --dport 22 -s 192.168.113.33 -j ACCEPT**
- **iptables -A INPUT -p tcp --dport 22 -s 192.168.113.0/24,192.168.114.0/24 -j DROP**

Iptables - basic usage

If you specify several source or destination networks, separate rules for each network will be created

- **iptables -L -nv**

```
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
   15  3000 ACCEPT     tcp  --  *      *       192.168.113.33       0.0.0.0/0           tcp dpt:22
    0     0 DROP       tcp  --  *      *       192.168.113.0/24     0.0.0.0/0           tcp dpt:22
    1    60 DROP       tcp  --  *      *       192.168.114.0/24     0.0.0.0/0           tcp dpt:22
```

Iptables - basic usage

To display rule numbers next to each firewall rule, use '--line-numbers'

- **iptables -L -nv --line-numbers**

```
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1       15  3000 ACCEPT     tcp  --  *      *       192.168.113.33       0.0.0.0/0           tcp dpt:22
2        0     0 DROP       tcp  --  *      *       192.168.113.0/24     0.0.0.0/0           tcp dpt:22
3        1    60 DROP       tcp  --  *      *       192.168.114.0/24     0.0.0.0/0           tcp dpt:22
```

Iptables - basic usage

To remove specific rule, use corresponding rule number

- **iptables -D INPUT 2**

- **iptables -D OUTPUT 14**

Iptables - basic usage

If you want to add new 'iptables' rule to specific rule number, use '-I' option followed by chain and rule number

- **iptables -I INPUT 4 -p icmp -j DROP**
- **iptables -L -nv --line-numbers**

```
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1       15  3000 ACCEPT     tcp  --  *      *       192.168.113.33       0.0.0.0/0           tcp dpt:22
2        0     0 DROP       icmp --  *      *       0.0.0.0/0            0.0.0.0/0
3        1    60 DROP       tcp  --  *      *       192.168.114.0/24     0.0.0.0/0           tcp dpt:22
4        0     0 DROP       tcp  --  *      *       192.168.113.0/24     0.0.0.0/0           tcp dpt:22
5        0     0 DROP       tcp  --  *      *       192.168.114.0/24     0.0.0.0/0           tcp dpt:22
6        0     0 DROP       tcp  --  *      *       192.168.115.0/24     0.0.0.0/0           tcp dpt:22
```

Iptables - basic usage

If you want to replace existing 'iptables' rule, use '-R' option followed by chain and rule number

- **iptables -R INPUT 4 -d 192.168.113.0/24 -p icmp -j DROP**
- **iptables -L -nv --line-numbers**

```
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1       15  3000 ACCEPT     tcp  --  *      *       192.168.113.33       0.0.0.0/0           tcp dpt:22
2        0     0 DROP       icmp --  *      *       0.0.0.0/0            0.0.0.0/0
3        1    60 DROP       tcp  --  *      *       192.168.114.0/24     0.0.0.0/0           tcp dpt:22
4        0     0 DROP       icmp --  *      *       0.0.0.0/0            192.168.113.0/24
5        0     0 DROP       tcp  --  *      *       192.168.114.0/24     0.0.0.0/0           tcp dpt:22
6        0     0 DROP       tcp  --  *      *       192.168.115.0/24     0.0.0.0/0           tcp dpt:22
```

Iptables - basic usage

Iptables record for each rule number of packets matched and number of bytes processed by the rule. To reset all counters, use '-Z' option followed by chain name

- **iptables -Z INPUT**

To reset single rule counters, add rule number after chain name

- **iptables -Z INPUT 3**

Iptables - basic usage

To delete all rules from specific chain, use '-F' option

- **iptables -F INPUT**
- **iptables -L -nv --line-numbers**

```
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
```

Iptables - basic usage

To delete ALL(!) firewall rules, run following commands

- **iptables -F**
- **iptables -Z**

Iptables - basic usage

To make your firewall rules persistent (rules will be active after machine reboot), install following package

- **apt-get install iptables-persistent**

Enable and start service after installation

- **systemctl enable netfilter-persistent.service**
- **systemctl start netfilter-persistent.service**

Iptables - basic usage

To make your firewall rules persistent (rules will be active after machine reboot), install following package

- **iptables-save > /etc/iptables/rules.v4**

If your system has IPv6 rules, then you have to use corresponding 'ip6tables' command

- **ip6tables-save > /etc/iptables/rules.v6**

Iptables - basic usage

With '**iptables**' you can redirect traffic from one port to another. Let's first start 'apache2' webserver

- **service apache2 start**

Open your server IP address in the browser, to verify web server is up and running, and reachable.

Iptables - basic usage

Now, let's add new rule, which will be redirecting tcp request from port 8080 to port 80

- **iptables -t nat -A PREROUTING -p tcp --dport 8080 -j REDIRECT --to-ports 80**

Now try to access your machine's IP address over port 8080.

Iptables - basic usage

For redirecting the traffic '**iptables**' uses '**nat**' table.

To view 'nat' table rules, use '-t' option followed by table name

- **iptables -L -nv -t nat --line-numbers**

```
Chain PREROUTING (policy ACCEPT 4 packets, 450 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 REDIRECT   tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:8080 redir ports 80

Chain INPUT (policy ACCEPT 3 packets, 120 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 4 packets, 240 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 4 packets, 240 bytes)
num   pkts bytes target     prot opt in     out     source               destination
```

Iptables - basic usage

Time for hands-on.

Let's start fake DNS server on your Kali machine.

DNS server will be listening on port 5353 and all DNS requests will be resolved as 192.168.11.11.

Be sure to set correct listening IP address

- **dnschef -i 192.168.XX.XX -p 5353 --fakeip 192.168.11.11**

```
(11:40:15) [*] Listening on an alternative port 5353
(11:40:15) [*] DNSChef started on interface: 192.168.113.153
(11:40:15) [*] Using the following nameservers: 8.8.8.8
(11:40:15) [*] Cooking all A replies to point to 192.168.11.11
```

Iptables - basic usage

Now create iptables rule, which will forward DNS requests from port 53 to your fake DNS server.

Now do DNS lookup against your Kali machine.

What is the result?

Iptables - basic usage

Let's create new rule, which will prevent your server from port scanning.

Firstly, from Kali server we run basic port scan against the target

- **nmap 10.XX.1.10**

```
Nmap scan report for 192.168.113.153
Host is up (0.0000050s latency).
Not shown: 998 closed ports
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http

Nmap done: 1 IP address (1 host up) scanned in 6.60 seconds
```

Iptables - basic usage

Now we create new 'iptables' chain

- **iptables -N port_scan**

Next step is to forward all TCP-SYN requests to new chain 'port_scan'

- **iptables -A INPUT -p tcp --syn -j port_scan**

Iptables - basic usage

Finally, we set the limits and 'drop' all packets above the limit

- **iptables -A port_scan -m limit --limit 1/s --limit-burst 3 -j RETURN**
- **iptables -A port_scan -j DROP**

Scan again your server with '**nmap**'

- **nmap 192.168.XX.XX**

CYBEXER TECHNOLOGIES

Apache2 hardening

Apache2 hardening

Apache2 web server

Apache2 hardening

By-default the Apache version and OS are shown in the response headers. This is a major security loophole exposing such details to the world and be used by hackers.

Let's see what information is readable

- **curl -I http://10.XX.1.20/**

```
# curl -I 192.168.115.245
HTTP/1.1 200 OK
Date: Thu, 11 Apr 2024 12:10:43 GMT
Server: Apache/2.4.52 (Ubuntu)
Last-Modified: Mon, 13 Mar 2023 11:29:11 GMT
ETag: "29af-5f6c66b8549cb"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html
```

Apache2 hardening

To hide those details, add the two lines in Apache config file

- **sed -i 's/^ServerTokens OS/ServerTokens Prod/' /etc/apache2/conf-enabled/security.conf**
- **sed -i 's/^ServerSignature On/ServerSignature Off/' /etc/apache2/conf-enabled/security.conf**
- **systemctl restart apache2**

After Apache2 is restarted, re-run curl command from previous slide and see results.

Apache2 hardening

By default, the directory listing for all files under web root directory is enabled if there is no index.

This allows hackers to view and analyze the files in your web server directory and maximize on the slightest available vulnerability to launch an attack.

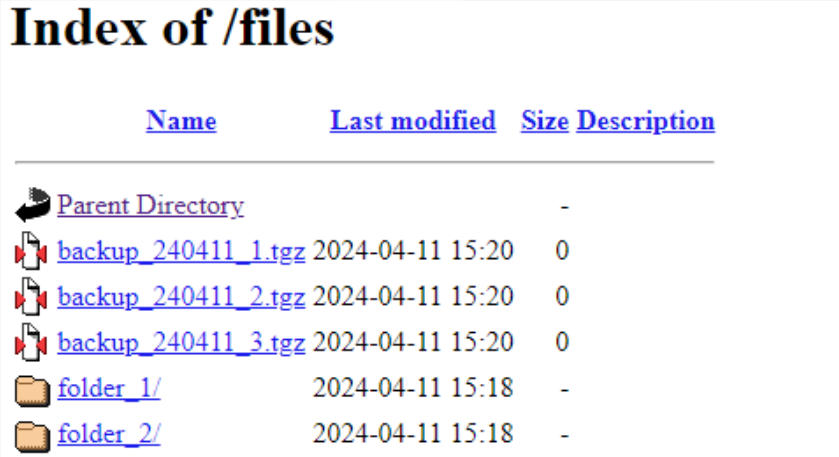
Apache2 hardening

Let's create new web-root sub-folder and mimic some file structure there

- **mkdir -p /var/www/html/files/folder_{1..8}**
- **touch /var/www/html/files/backup_`date '+%y%m%d'`_{1,2,3}.tgz**

Browse you web server

- **http://10.XX.1.10/files/**

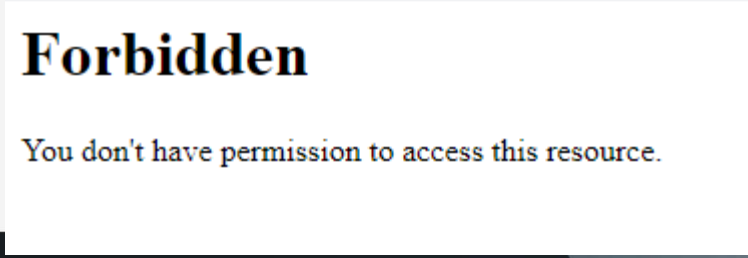


Apache2 hardening

Next step is to disable directory listing, restart Apache2 web server and check the results

- **sed -i 's/Options Indexes FollowSymLinks/Options -Indexes -FollowSymLinks/' /etc/apache2/apache2.conf**
- **systemctl restart apache2**

Now view **http://10.XX.1.10/files/** in your browser



Apache2 hardening

By default, Apache2 does not have HTTPS configured.

Run following commands to enable HTTPS server

- **a2enmod ssl**
- **a2ensite default-ssl.conf**
- **systemctl restart apache2**

Apache2 hardening

To verify Apache SSL/TLS setting, we will use tool called 'sslscan'. Let's install it on Kali system

- **apt-get -y install sslscan**

```
# apt-get -y install sslscan
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  sslscan
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 1,519 kB of archives.
After this operation, 4,878 kB of additional disk space will be used.
Get:1 https://kali.download/kali kali-rolling/main amd64 sslscan amd64 2.1.3
Fetched 1,519 kB in 1s (1,352 kB/s)
```

Apache2 hardening

Next step is to scan Apache2 and review the results

- **sslscan 10.XX.1.10**

```
  SSL/TLS Protocols:
SSLv2     disabled
SSLv3     disabled
TLSv1.0   disabled
TLSv1.1   disabled
TLSv1.2   enabled
TLSv1.3   enabled
```

```
  Supported Server Cipher(s):
Preferred TLSv1.3  128 bits  TLS_AES_128_GCM_SHA256         Curve 25519 DHE 253
Accepted  TLSv1.3  256 bits  TLS_AES_256_GCM_SHA384         Curve 25519 DHE 253
Accepted  TLSv1.3  256 bits  TLS_CHACHA20_POLY1305_SHA256   Curve 25519 DHE 253
Preferred TLSv1.2  256 bits  ECDHE-RSA-AES256-GCM-SHA384    Curve 25519 DHE 253
Accepted  TLSv1.2  256 bits  DHE-RSA-AES256-GCM-SHA384      DHE 2048 bits
Accepted  TLSv1.2  256 bits  ECDHE-RSA-CHACHA20-POLY1305    Curve 25519 DHE 253
Accepted  TLSv1.2  256 bits  DHE-RSA-CHACHA20-POLY1305      DHE 2048 bits
Accepted  TLSv1.2  256 bits  DHE-RSA-AES256-CCM8           DHE 2048 bits
Accepted  TLSv1.2  256 bits  DHE-RSA-AES256-CCM            DHE 2048 bits
Accepted  TLSv1.2  256 bits  ECDHE-ARIA256-GCM-SHA384      Curve 25519 DHE 253
```

Apache2 hardening

Based on your security requirements, adjust following options in your Apache2 configuration

SSLCipherSuite

SSLProtocol

SSLHonorCipherOrder

Testing prior to production use is highly recommended!

Apache2 hardening

To generate own self-signed certificate run following command:

- **openssl req -x509 -nodes -days 3650 -newkey rsa:2048 -keyout /etc/ssl/private/web.key -out /etc/ssl/certs/web.crt --subj '/C=XX/O=Cyber-X Ltd./OU=IT/CN=server' -addext "subjectAltName = DNS:`hostname`,IP:`hostname -I |awk '{print $1}'`"**

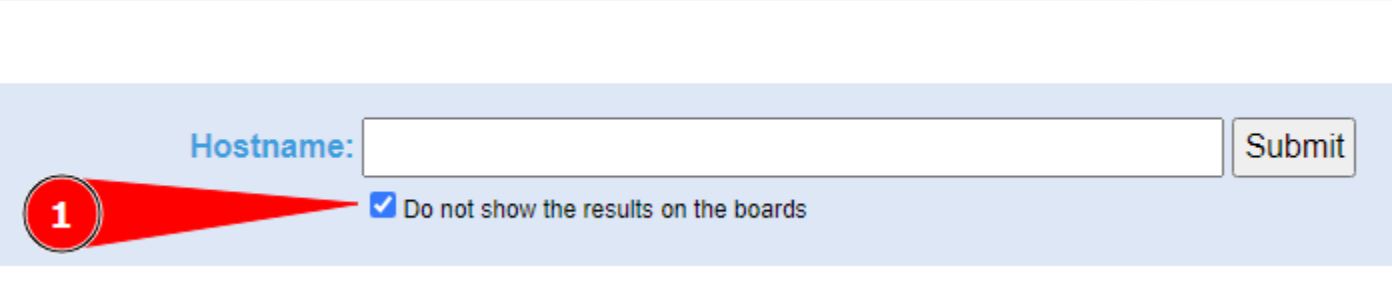
```
O=Cyber-X Ltd./OU=IT/CN=server' -addext "subjectAltName = DNS:`hostname`,
Generating a RSA private key
.....................................................................
.....................................................+++++
.........................+++++
writing new private key to '/etc/ssl/private/web.key'
-----
```

Apache2 hardening

To check publicly reachable web server's SSL settings, use following site:

- **https://www.ssllabs.com/ssltest/**

Before submitting, enable "Do not show" checkbox, to hide your scan from the results page.



Apache2 hardening

To add additional security to Apache2 web server you can implement some useful HTTP headers:

- **Header always set X-Content-Type-Options: "nosniff"**
- **Header always set X-Frame-Options: "sameorigin"**
- **Header always set X-XSS-Protection: 1**
- **Header always set Content-Security-Policy: "default-src 'none'; script-src 'none'; style-src 'self'; img-src 'self'"**
- **Header always set Strict-Transport-Security "max-age=63072000; includeSubDomains; preload"**

To enable "Header" directive usage, you must enable "headers" module and restart Apache server

- **a2enmod headers ; systemctl restart apache2**

Apache2 hardening

The **Strict-Transport-Security header** is a security enhancement that restricts web browsers to access web servers solely over HTTPS. This ensures the connection cannot be establish through an insecure HTTP connection which could be susceptible to attacks.

The **X-XSS-Protection header** is designed to enable the cross-site scripting (XSS) filter built into modern web browsers.

Apache2 hardening

The **X-Frame-Options header** is designed to prevent site content embedded into other sites.

The **Content-Security-Policy header** allows you to restrict which resources (such as JavaScript, CSS, Images, etc.) can be loaded, and the URLs that they can be loaded from.

The **X-Content-Type-Options header** ensures that user agents do not attempt to guess the format of the data being received.

Apache2 hardening

To check HTTP headers, you can use 'humble' tool. Let's install it on the Kali machine

- **apt-get install humble**

```
# apt-get install humble
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-filelock python3-fpdf python3-tldextract
The following NEW packages will be installed:
  humble python3-filelock python3-fpdf python3-tldextract
0 upgraded, 4 newly installed, 0 to remove and 0 not upgraded.
Need to get 309 kB of archives.
After this operation, 1,504 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

Apache2 hardening

Now, let's run humble HTTP check tool against our web Apache server

- **humble -u http://10.XX.1.10**

```
Strict-Transport-Security
Tell browsers that it should only be accessed using HTTPS, instead of using HTTP.
Ref: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Sec

X-Content-Type-Options
Indicate that MIME types in the "Content-Type" headers should be followed.
Ref: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Optio

X-Permitted-Cross-Domain-Policies
Limit which data external resources (e.g. Adobe Flash/PDF documents), can access on
Ref: https://owasp.org/www-project-secure-headers/#div-headers

X-Frame-Options
Prevents clickjacking attacks, limiting sources of embedded content.
Ref: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
```

```
Analysis done in 0.4 seconds! (changes with respect to the last

  Missing headers:              13 (First Analysis)
  Fingerprint headers:          1 (First Analysis)
  Deprecated/Insecure headers:  2 (First Analysis)
  Empty headers:                0 (First Analysis)

  Findings to review:           16 (First Analysis)
```

Apache2 hardening

After adding required HTTP headers, re-run 'humble' and see the results

- **humble -u http://10.XX.1.10**

```
[5. Browser Compatibility for Enabled HTTP Security Headers]

 Content-Type: https://caniuse.com/?search=Content-Type
 Content-Security-Policy: https://caniuse.com/?search=contentsecuritypolicy2
 ETag: https://caniuse.com/?search=ETag
 Strict-Transport-Security: https://caniuse.com/?search=Strict-Transport-Security
 X-Content-Type-Options: https://caniuse.com/?search=X-Content-Type-Options
 X-Frame-Options: https://caniuse.com/?search=X-Frame-Options
 X-XSS-Protection: https://caniuse.com/?search=X-XSS-Protection
```

```
Analysis done in 0.41 seconds! (changes with respect to the

  Missing headers:                9 (-4)
  Fingerprint headers:            1 (0)
  Deprecated/Insecure headers:    5 (+3)
  Empty headers:                  0 (0)

  Findings to review:             15 (-1)
```

Apache2 hardening

Another tool for HTTP header check is 'shcheck.py'. Let's install on Kali machine

- **pip3 install shcheck**

```
 replacement is to use pip for package installation.. Discussion c
Collecting shcheck
  Downloading shcheck-1.5.9-py3-none-any.whl.metadata (2.6 kB)
Downloading shcheck-1.5.9-py3-none-any.whl (23 kB)
Installing collected packages: shcheck
Successfully installed shcheck-1.5.9
WARNING: Running pip as the 'root' user can result in broken permi
l environment instead: https://pip.pypa.io/warnings/venv
```

Apache2 hardening

Now run "shcheck.py" against your Apache2 web server

- **shcheck.py http://10.XX.1.10**

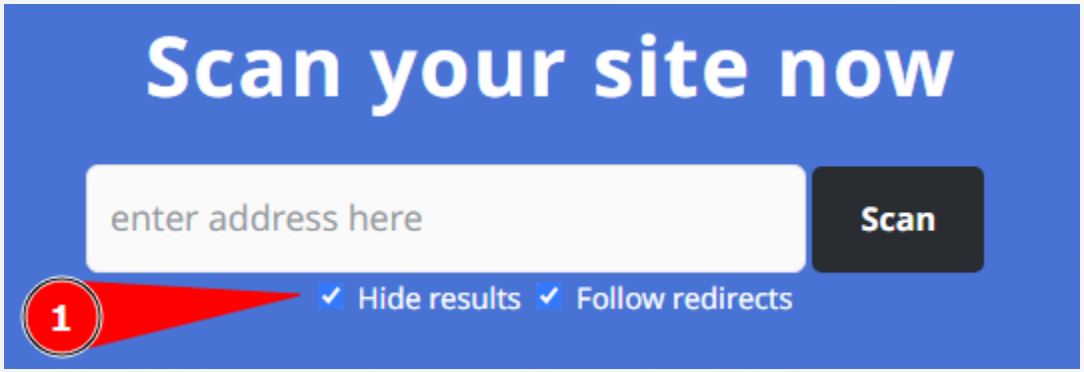
```
-------------------------------------------------------
 Simple tool to check security headers on a webserver
=======================================================

[*] Analyzing headers of http://192.168.115.245
[*] Effective URL: http://192.168.115.245
[*] Header X-XSS-Protection is present! (Value: 1)
[*] Header X-Frame-Options is present! (Value: sameorigin)
[*] Header X-Content-Type-Options is present! (Value: nosniff)
[*] Header Strict-Transport-Security is present! (Value: max-age=63072000; in
[*] Header Content-Security-Policy is present! (Value: default-src 'none'; sc
[!] Missing security header: Referrer-Policy
[!] Missing security header: Permissions-Policy
[!] Missing security header: Cross-Origin-Embedder-Policy
[!] Missing security header: Cross-Origin-Resource-Policy
[!] Missing security header: Cross-Origin-Opener-Policy
-------------------------------------------------------
```

Apache2 hardening

To check HTTP security headers for public web site, you can use following site:

- **https://securityheaders.com/**

Be sure to enable option "Hide results", this will prevent your site appear in scanned site list



CYBEXER TECHNOLOGIES